

✓ Genetic Algorithms for the Variational Quantum Eigensolver (GA VQE)

Nicholas Pittman

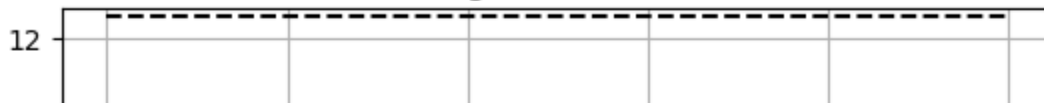
Executive Summary

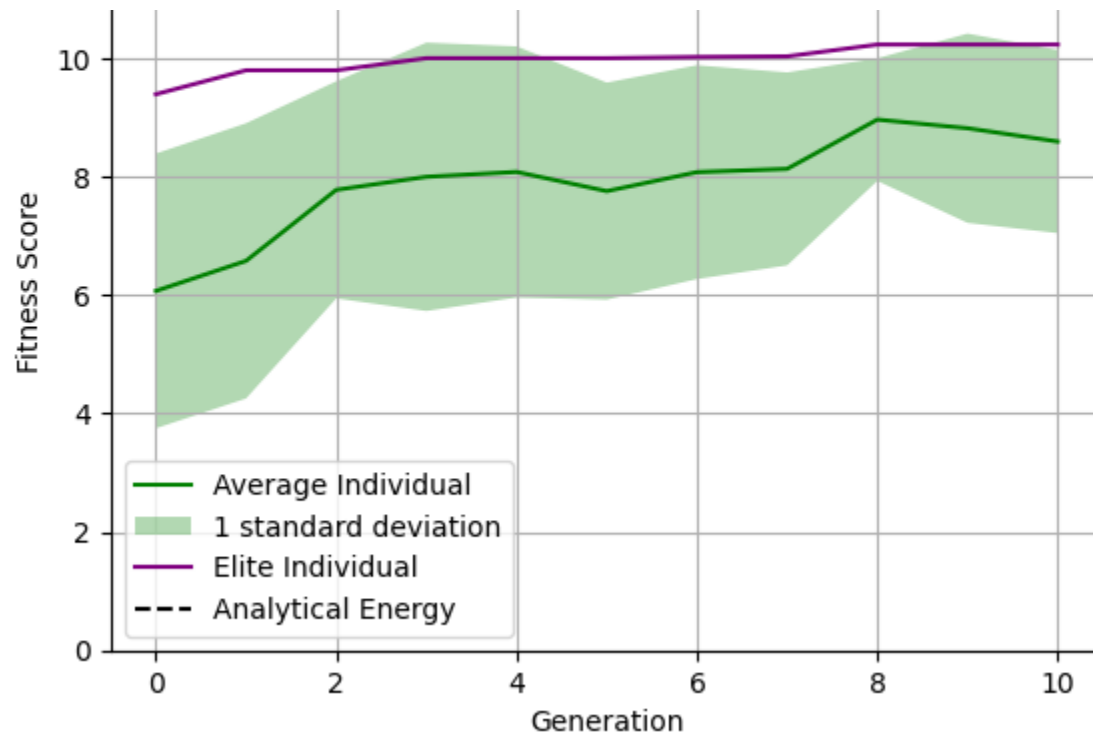
We develop from complete scratch a genetic algorithm that selects/optimizes quantum computing circuits for use in a variational eigensolver, the goal of which is to approximate the lowest eigenvalues/vectors of a Hermitian matrix (usually a physical Hamiltonian).

This work is a proof-of-concept demonstration of the topic, but is in line with current research such as <https://arxiv.org/abs/2110.07441> and https://link.springer.com/chapter/10.1007/978-981-96-4596-1_22 in terms of its scope (which I did not know of/reference when coding this project). I was inspired to investigate this implementation as a suggested term project for the course AI for Physicists at TU Delft in the Netherlands (<https://www.eliskagreplova.com/ai-for-physicists-ap3751>).

We successfully develop and test the algorithm on a simple physical Hamiltonian of the Ising model, and reproduce exact results for $n = 2, 4$ to verify the algorithm functions sensibly. We then apply it up to $n = 10$ qubits (for reference, a 20 qubit system overwhelms available RAM) to test it under stress, and we see a trend of approach towards the exact eigenvalue and by the 10th generation we are within 15% of the exact eigenvalue, which is intuitively reasonable given the immense complexity of the search space for possible 10-qubit topologies. The result figure from the end of the notebook is produced below:

Average and Elite Fitness vs. Generation
for Genetically-Produced Quantum Circuits Preparing Ground State
of the $n = 10$ Ising Model (with 1σ error bars)





For this survey implementation in a notebook environment (which is perhaps a poor choice by the author), we were extremely constrained by run time limits to only 10 generations each comprised of ~30-50 individuals (run time ~5 minutes); as such, we do not expect nor desire extreme accuracy or convergence. Implementation on legitimate hardware would allow for substantially higher force to be applied to the genetic search and for longer periods of time, and would presumably yield convergence. The algorithm passed available sanity checks and exhibited desirable behavior up to 10 qubits, implying it would perform well if given the opportunity to run with more power.

The algorithm and procedure is relatively modular, and can be adjusted to use different quantum gate sets or different fitness functions for a variety of desired uses in quantum state preparation.

Attributions

All coding and commentary were written, by hand, by the author, except for some string formatting and Qiskit circuit

formatting instructions that were done by ChatGPT. The author also used the IBM Qiskit tutorial on the VQE (<https://quantum.cloud.ibm.com/learning/en/courses/quantum-diagonalization-algorithms/vqe>) for background information on the method.

Background

The goal of a variational eigensolver is to numerically approximate the ground state and corresponding energy for some Hamiltonian matrix H using the variational approach: that is, guess-and-check quantum states, compute the expectation value of H in that state, and hunt for the state that yields the minimum output. The variational principle says that this is the ground state of the system. This method is applicable not just to quantum mechanical Hamiltonians, but any Hermitian matrix.

In our case, we will use Hamiltonians in a simple class of physical systems: a transverse-field Ising model, which is a chain of n interacting spins (with index i ranging from 0 to $n - 1$) with nearest-neighbor interactions with strength J , immersed in an external magnetic field of strength h :

$$H = -J \sum_{i=0}^{n-1} Z_i Z_{i+1} - h \sum_{i=0}^n X_i$$

where i indicates that the Pauli operator acts on the i th element of the chain.

The Classical Eigensolver Algorithm

The classical work flow would involve guessing the form of the ground state as a function of some tunable parameters θ , i.e. $|\psi(\theta_i)\rangle$ where $i = 0 \dots N - 1$ has up to as many entries as the number of qubits. Then, we need to compute the expectation value of this N -dimensional vector with respect to H , i.e. the matrix inner product $\langle \psi(\theta_i) | H | \psi(\theta_i) \rangle$. This multiplication involves N^2 steps. After computing this one matrix inner product, we will adjust θ_i and compute the inner product again, adjusting using some classical optimization method such as gradient descent. In other words, this $\mathcal{O}(N^2)$ operation must be done at *each* step of the classical minimization procedure that searches for θ_i .

Quantum Speedup

In the quantum version, this $O(N^2)$ step process is simplified down to making a measurement on the quantum circuit that outputs $\langle H \rangle_{|\psi\rangle}$. If we get lucky (and in many physical applications in chemistry etc. there is reason to believe we will be), H will be relatively sparse, and composed of a relatively small number of Pauli operators (or at least, a reasonable number of operator sets that mutually commute with each other, so we can measure them simultaneously without having to re-prepare the circuit) that can be measured easily on the quantum computer. In this way, the $O(N^2)$ -step process can be reduced to a series of measurements.

Genetic Algorithm

The genetic algorithm is a machine learning parameter-update algorithm one can employ for essentially any optimization task that you want to optimize performance on- for example, to find a function that minimizes a loss function, to find the weights of a neural network that optimize accuracy of the $in \rightarrow out$ prediction map, etc.

Each single guess/attempt at the solution will comprise an "individual". We initialize a set of individuals with random-ish properties (e.g. guess random weights W in the neural net problem). (In the computational implementation, the population a list of user-defined classes, with each class representing an individual. The individual class will then have attributes such as its genome and performance on the task at hand, known as its "fitness".)

We then have the individuals "compete" in a Darwinian "tournament" step, and only keep the survivors (i.e. the ones with better fitness). We can then have these survivors generate the next generation by crossing over genes in a simulated reproductive step (i.e. a child has genes 1-3 from parent 1, and 4-10 from parent 2), with the addition of a mutation step that can randomly change a gene with some probability. The children then become the parents in the population, and we begin again. This evolutionary process ensures only better-fitting individuals (at least in the local minimum sense) are produced in subsequent generations; the mutation step helps promote exploration beyond local minima.

In summary, the steps of a genetic algorithm are:

- First time only: initialize population of individuals with random attributes
- 1. Have individuals compete in tournament, discard losers
- 2. Have winners generate children via crossover

2. Have winners produce children via crossover
3. Act randomly on the children by mutation
4. Set population = children
5. Repeat

Genetic Algorithm Applied to Quantum Circuits

Each individual in our problem will be a quantum circuit, with genes being their gate topology. Circuit designs will be randomly initialized, "compete" in a Darwinian "tournament" from which we will keep the winners. We will then have these winners "reproduce" by splicing their genes onto one another and mutating them (in the form of random gate deletions, additions, or swaps). This will help us locate the circuit that prepares the ground state of a given Hamiltonian rather rapidly.

In addition to the discrete choices of topology we must make for each individual, there will also be continuous choices in the form of the rotation parameters θ_i of any $R_y(\theta_i)$ gates implemented on any qubits (i.e. a θ rotation performed about the y -axis in phase space). We will NOT incorporate the continuous parameters θ_i into the genome- the genome will purely index the circuit topology, and we will rely on a classical optimizer to locate the minimum θ_i for each. Of course, this will involve classical optimizations that will reduce the quantum speedup, but the search over the *space of circuit topologies* will be handled and sped up by the algorithm.

The fitness function we will operate on is the following (user-chosen) composite measure of how desirable a circuit is:

$$F = -E - \lambda N_{\text{gates}} - \lambda_y N_{R_y}$$

i.e. the fitness is equal to the negative of the ground state energy (we promote lower energy solutions), minus terms involving the number of gates weighted by some coefficients. We will penalize the number of R_y gates especially hard, given that they introduce the need for classical optimizations of their corresponding θ_i .

We reiterate that the fitness above is *user-chosen* and can be tweaked to measure, say, the fidelity of the states outputted by the circuit relative to some desired state, if one is interested in state preparation from a circuit rather than Hamiltonian expectation values.

Benefits of the Genetic Algorithm

The genetic algorithm has the distinct benefit of being a gradient-free form of optimization- it can be easily applied to discrete problems with no notion of a "good direction" (that is present in continuous tasks). This also helps the algorithm avoid accidental convergence local minima, an issue to which gradient-based optimizations are susceptible.

Quantum computing algorithms are further susceptible to having wide, featureless landscapes of parameter space known as *barren plateaus* (see <https://arxiv.org/abs/2405.00781>); the genetic algorithm introduces random modifications and maintains a high number of varying individuals to simultaneously explore much of the landscape of parameters simultaneously, and converges on favorable regions as the population undergoes crossover mutations linking favorable individuals to each other.

Limitations

There are obstructions to the efficiency of the purported quantum "speedup" of this process, however- for instance, quantum computation introduces the possibility of errors into the computation; the expectation value of a matrix must be determined from the *statistics of many repeated measurements*- if an extremely high degree of accuracy is demanded, the number of repeated measurements may outweigh the efficiency gains of avoiding computing the matrix inner product.

That being said, for tasks where finding *approximate* ground states/energies is of interest, the need for excessive measurement counts can be greatly reduced, and the algorithm at hand can provide a substantial speedup.

✓ Two-Qubit Ising Model- Getting used to Qiskit.

Prior to this project, I had never interacted with Qiskit before, so this case will allow us to manually write the Hamiltonian and learn how everything works. We will move to more modular coding styles later on.

This case also has the benefit of providing an exact answer and known optimal circuit, so we can sanity check our later algorithm against this (indeed it successfully reproduces the circuit and energy below).

✓ Defining the Hamiltonian

Defining the Hamiltonian

Can be done using numpy arrays, or built-in Qiskit modules like SparsePauliOp.

```
1 ##Uncomment to install needed dependencies.
2 %pip install qiskit
3 %pip install pylatexenc
```

```
Requirement already satisfied: qiskit in /usr/local/lib/python3.12/dist-packages (2.4.2)
Requirement already satisfied: rustworkx>=0.15.0 in /usr/local/lib/python3.12/dist-packages (from qiskit) (0.15.0)
Requirement already satisfied: numpy<3,>=1.21 in /usr/local/lib/python3.12/dist-packages (from qiskit) (2.0.2)
Requirement already satisfied: scipy>=1.5 in /usr/local/lib/python3.12/dist-packages (from qiskit) (1.16.3)
Requirement already satisfied: dill>=0.3 in /usr/local/lib/python3.12/dist-packages (from qiskit) (0.3.8)
Requirement already satisfied: stevedore>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from qiskit) (5.2.0)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from qiskit) (4.12.2)
Requirement already satisfied: pylatexenc in /usr/local/lib/python3.12/dist-packages (2.10)
```

```
1 import scipy
2 import numpy as np
3 import matplotlib
4 from matplotlib import pyplot as plt
5 import pandas as pd
6 import random
7 import copy
8
9 import qiskit
10 from qiskit.quantum_info import SparsePauliOp
11 from qiskit.circuit.library import HGate, XGate, CXGate, RYGate
```

```
1 ##Interaction and magnetic field strengths: (POSITIVE by convention)
2 J = 1.0
3 h = 1.0
```

```
1 # ###Numpy approach: (we will not use this)
2 # I = np.eye(2) #2x2 Identity
3
```

```

4 # X = np.array([
5 #     [0,1],
6 #     [1,0]])
7
8 # Z = np.array([
9 #     [1,0],
10 #     [0,-1]])
11
12 # H = (
13 #     -J*np.kron(Z,Z)
14 #     -h*np.kron(X,I)
15 #     -h*np.kron(I,X))
16
17 # print(H)

```

```

1 ###Qiskit approach (more modular)
2
3 H = SparsePauliOp.from_list([
4     ("ZZ",-J),
5     ("XI",-h),
6     ("IX",-h)])
7
8 print(H)

```

```

SparsePauliOp(['ZZ', 'XI', 'IX'],
               coeffs=[-1.+0.j, -1.+0.j, -1.+0.j])

```

✓ Finding exact ground state and energy for 2-qubit Ising using `np.linalg` (for comparison/sanity checking)

```

1 eigvals, eigvecs = np.linalg.eigh(H)
2
3 ground_energy = eigvals[0]
4 ground_state = eigvecs[:,0]
5 print("Ground state:", ground_state)

```

```
6 print("Ground state energy =", ground_energy)
```

```
Ground state: [0.60150096+0.j 0.37174803+0.j 0.37174803+0.j 0.60150096+0.j]
Ground state energy = -2.23606797749979
```

✓ Using a quantum circuit to prepare states

The above calculation was done without any mention of a quantum circuit preparing the state, something we now introduced.

The known topology of the circuit preparing the ground state of the 2-qubit ising model is a ctrl-X (CNOT) gate preceded by two rotation operators with parameter vector `theta = [theta[0], theta[1]]` that controls the strength of the rotation on qubit zero and 1. This was an AI-suggested ansatz that was confirmed a posteriori by comparison with the above exact results.

We can then extract the output statevector these gates yield by using

`qiskit.quantum_info.Statevector.from_instruction`, and then extracting the `data` attribute of the produced (abstract state class) `psi`.

We can then compute the energy of this guessed state, again either using numpy operations (vector daggered, dotted into Hamiltonian, dotted into vector) or built-in Qiskit methods.

The below cells, respectively, create a quantum circuit for a fixed array of θ , define an energy function that calculates the energy associated to a circuit prepared with such an array (this function uses the circuit-creating function within it), and lastly employ a scipy optimizer to minimize with respect to θ_i .

```
1 def make_circuit(theta):
2     #Intakes two parameters corresponding to initial rotation gates.
3     #Returns a circuit object
4
5     qc = qiskit.QuantumCircuit(2) #Initializes quantum circuit with 2-qubit register
6     qc.ry(theta[0],0) #Adds a rotation gate to qubit zero, of angle theta[0]
7     qc.ry(theta[1],1)
```

```

8     qc.cx(0,1) #Adds a controlled x gate with control qubit 0 and target qubit 1
9
10    return qc
11
12    test_theta = [.1,.1]
13
14    test_qc = make_circuit(test_theta)
15    test_psi = qiskit.quantum_info.Statevector.from_instruction(test_qc)
16    test_energy_value = test_psi.expectation_value(H)
17
18    print("Energy corresponding to prepared psi with theta parameter vector", test_theta, ": ", test_ener

```

Energy corresponding to prepared psi with theta parameter vector [0.1, 0.1] : (-1.1048042930042332+0j)

```

1 from scipy.optimize import minimize
2 theta0 = np.random.randn(2)
3
4 def energy(theta):
5     qc = make_circuit(theta) #Makes circuit with those input values
6     psi = qiskit.quantum_info.Statevector.from_instruction(qc) #Computes outputted state
7     # state = psi.data; H_matrix = H.to_matrix(); expval_energy = np.real(state.conj().T @ H_matrix @ s
8     expval_energy = psi.expectation_value(H) #Built-in expectation value attribute of qiskit objects.
9     return np.real(expval_energy) #Returns expval of Hamiltonian on this state (applies np.real to ensu
10
11 test_energy = energy([.1,.1])
12 print("Test energy:", test_energy)

```

Test energy: -1.1048042930042332

```

1 result = minimize(
2     energy,
3     theta0,
4     method="COBYLA")
5
6 print("Result:", result)

```

Result: message: Return from COBYLA because the trust region radius reaches its lower bound.
success: True

```

success: True
status: 0
  fun: -2.236067975301606
   x: [ 1.571e+00  1.107e+00]
  nfev: 52
 maxcv: 0.0

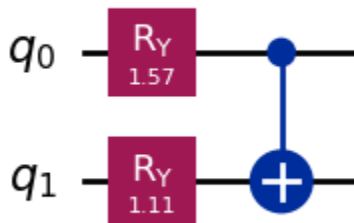
```

✓ Plotting circuit that prepares the numerically-computed ground state

```

1 best_qc = make_circuit(result.x) #extracts the parameters of the optimized ground state, which is store
2 best_qc.draw("mpl") #Draws circuit (requires installing matplotlib dependency and restarting Kernel)

```



Conclusion for 2-qubit Ising model:

We see the numerical optimization of the Qiskit circuit yielded a ground state energy of -2.23, in agreement with the analytical result.

✓ n -qubit Transverse-Field Ising Model

With $n > 2$ qubits we will take the opportunity to get used to more reproducible, loop-based methods for generating the Pauli strings in the Hamiltonian (in the 2 qubit case we hard-coded them).

In practice, this is achievable by joining together letters using the `"".join(my_array)` to connect the strings into one string, packaging the result into a list of the form `[XIZZ, coefficient]`, putting each of these into an array called "terms", and then passing that entire array to `SparsePauliOp.from_list(terms)`.

Function that generates n -qubit Ising Hamiltonian: ($n = 10$ in examples)

```

1 def transverse_ising_hamiltonian(n_qubits, J, h):
2     terms = [] #Will eventually be used to generate the SparsePauliOp. Each entry will be a 2-array of
3
4     for i in range(n_qubits-1): #Indices go from 0 to n_qubits-2, so the final pauli string has Z_{i=n-
5         pauli_string_array = ["I"]*n_qubits #Initializes the Pauli string as an array which we will lat
6         pauli_string_array[i] = "Z"
7         pauli_string_array[i+1] = "Z"
8         joined_string = "".join(pauli_string_array) #Creates string of the form "IIIZZIIII"
9         terms.append([joined_string, -J]) #Adds this term to the list "terms"
10    for i in range(n_qubits): #Handles the X terms
11        pauli_string_array = ["I"]*n_qubits #Initializes the Pauli string as an array which we will lat
12        pauli_string_array[i] = "X"
13        joined_string = "".join(pauli_string_array) #Creates string of the form "IIIZZIIII"
14        terms.append([joined_string, -h]) #Adds this term to the list "terms"
15
16    return SparsePauliOp.from_list(terms) #Returns a SparsePauliOp object from qiskit, with terms like
17
18 my_hamiltonian = transverse_ising_hamiltonian(10,1,1)
19 print(my_hamiltonian.to_matrix())
20 ###Exact ground state of system:
21 eigvals, eigvecs = np.linalg.eigh(my_hamiltonian)
22 ground_energy = eigvals[0]
23 ground_state = eigvecs[:,0]
24 print("Ground state:", ground_state)
25 print("Ground state energy =", ground_energy)

```

```

[[-9.+0.j -1.+0.j -1.+0.j ...  0.+0.j  0.+0.j  0.+0.j]
 [-1.+0.j -7.+0.j  0.+0.j ...  0.+0.j  0.+0.j  0.+0.j]
 [-1.+0.j  0.+0.j -5.+0.j ...  0.+0.j  0.+0.j  0.+0.j]
 ...

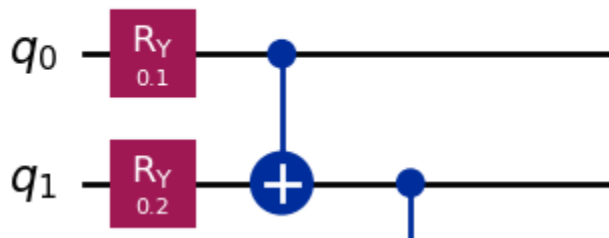
```

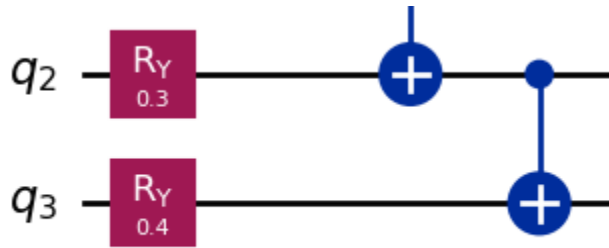
```
[ 0.+0.j  0.+0.j  0.+0.j ... -5.+0.j  0.+0.j -1.+0.j]
[ 0.+0.j  0.+0.j  0.+0.j ...  0.+0.j -7.+0.j -1.+0.j]
[ 0.+0.j  0.+0.j  0.+0.j ... -1.+0.j -1.+0.j -9.+0.j]]
Ground state: [-0.29339911+0.j -0.14835658+0.j -0.09318807+0.j ... -0.09318807+0.j
-0.14835658+0.j -0.29339911+0.j]
Ground state energy = -12.381489999654757
```

✓ Elementary circuit preparer for n -qubit circuit

This will have a fixed list of gates for now, R_y gates followed by a layer of controlled-X gates. We will do this simple case first to check that the linear algebra works properly, and we will later make a more complicated version capable of preparing circuits of any gate topology.

```
1 def prepare_circuit(n_qubits, theta):
2     """Generates a circuit with n Ry gates of rotation parameter theta[i], followed by entanglement by
3     qc = qiskit.QuantumCircuit(n_qubits) #Initializes quantum circuit with 2-qubit register
4     for i in range(n_qubits):
5         qc.ry(theta[i],i) #Adds a rotation gate to qubit zero, of angle theta[0]
6     for i in range(n_qubits-1):
7         qc.cx(i,i+1) #Adds a controlled x gate with control qubit 0 and target qubit 1
8     return qc
9
10
11 """Test- make a 4-qubit circuit with random
12 test_qc = prepare_circuit(4,[.1,.2,.3,.4])
13 test_qc.draw("mpl")
```





- ✓ Using minimization of quantum circuit-prepared states to find eigenvalues/eigenvectors of Hamiltonian:

```

1 from scipy.optimize import minimize
2
3 def energy(theta_array): #Given a fixed circuit geometry "prepare_circuit" and an explicit value for a
4     qc = prepare_circuit(n_qubits, theta_array)
5     psi = qiskit.quantum_info.Statevector.from_instruction(qc) #Computes the state prepared by that cir
6     expval_energy = psi.expectation_value(H) #Built-in expectation value attribute of qiskit objects.
7     return np.real(expval_energy) #Returns expval of Hamiltonian on this state (applies np.real to ensu
8
9 ###Test calculation- find the energy of a random choice of circuit theta parameters and Hamiltonian.
10 n_qubits = 10
11 H = transverse_ising_hamiltonian(n_qubits,J=1,h=1)
12
13 test_energy = energy(theta_array = [0,.1,.2,.3, 0,.1,.2,.3, 0,.1])
14 print("Test energy:", test_energy)
15
16
17 ###Test calculation- use scipy's minimize to find theta values corresponding to the minimum energy (at
18 result = minimize(
19     energy, #Assumes this is a function with one argument, which the optimizer will output as "x". Real
20     np.random.uniform(
21         -np.pi,
22         np.pi,
23         size=n_qubits), #Initial guess
24     method="COBYQA")

```

```

24     method="COBYLA")
25
26 print("Resulting theta array:", result.x, "and corresponding energy:", result.fun)
27 print("compared to the analytical answer of ground state energy (computed a cell above via diagonalizat
28
29 qc_result = prepare_circuit(n_qubits, result.x)
30 print("Circuit preparing ground state:")
31 print(qc_result)

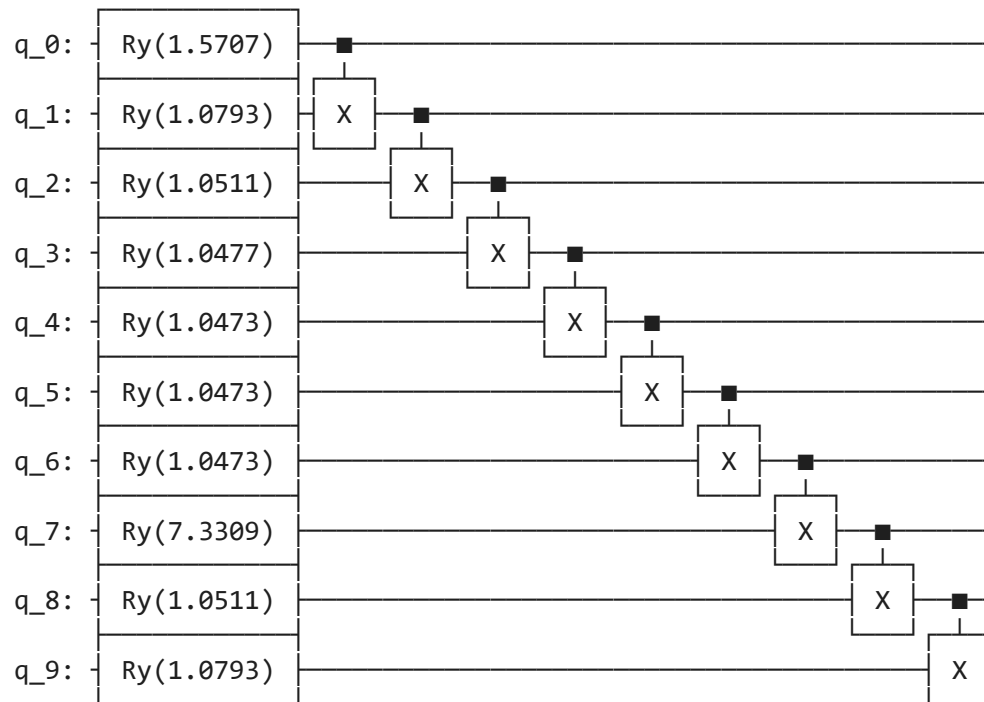
```

Test energy: -9.112741325954671

Resulting theta array: [1.57072136 1.07927153 1.05112883 1.04768911 1.04725072 1.04728684

1.0472762 7.3308601 1.05111423 1.07933744] and corresponding energy: -12.234201155729693

compared to the analytical answer of ground state energy (computed a cell above via diagonalization) of -12.
Circuit preparing ground state:



- ✓ Adding multiple trials to avoid local minima

```

1  ###Checks by iterating over a couple of trials that we have not just found a local minimum of energy
2  best_E = np.inf
3  best_result = None
4  num_trials = 5
5
6  for trial in range(num_trials):
7      theta0 = np.random.uniform(
8          -np.pi,
9          np.pi,
10         size=10)
11
12     result = minimize(
13         energy,
14         theta0,
15         method="COBYLA")
16
17     if result.fun < best_E:
18         best_E = result.fun
19         best_result = result
20
21 print("Best theta array:", best_result.x, "and corresponding energy:", best_result.fun)
22 print("compared to the analytical answer of ground state energy (computed a cell above via diagonalizat

```

```

Best theta array: [1.57084759 1.07927927 1.05111556 1.04765899 1.04722581 1.04719575
1.04721702 1.04768278 1.0511186 1.07927917] and corresponding energy: -12.234201162609184
compared to the analytical answer of ground state energy (computed a cell above via diagonalization) of -12.

```

Conclusion for n -qubit optimization:

The ground state energy computed by minimizing $\langle H \rangle$ by varying θ in the quantum circuit agreed with the analytical case to within 1% (-12.23 vs -12.38 ground state energies).

✓ PART 2- The Genetic Algorithm

In this genetic algorithm, each individual will be a quantum circuit, with the unique geometry/gate pattern of that individual comprising its "genome". We remind the reader that we will *not* include the θ_i as part of the genome, since we will have to classically re-optimize these parameters each time we generate a new circuit anyways. In other words, the *true* search that the genetic algorithm is performing is *not for the parameters θ_i , but rather the pattern/depth of gates a circuit should have* that most efficiently prepares the quantum state corresponding to the ground state of H .

The algorithm will pit different individuals against each other- ones with a layer of R_y gates at the beginning, ones with additional layers of CNOT/CX gates, additional layers, etc. Whichever one has higher "fitness" as measured by the above-defined fitness function (relating to the resulting classically-optimized ground state energy corresponding to that circuit geometry), will be deemed a survivor worthy of generating the next generation; from the survivors we will mix the genomes, i.e. splicing the gates of successful circuits onto one another, and then mutating circuits by adding or deleting random gates, or replacing gates with other gates randomly. This will ensure adequate exploration of the parameter search space beyond the starting population.

The below functions will fully modular-ize the above code, allowing for the random creation of gates and automatic handling of circuits with a differing number of gates/geometry. Functions are described in comments next to their definitions in the code.

Comments on Code/Algorithm Design:

Functions relating to R_y gates:

Many functions below deal with detecting the number of R_y gates. This unique focus on these gates is due to the fact that we *must* feed in some array of θ_i parameters when creating a circuit object, even if we will later feed in initial guesses and optimize with respect to them. As such, we by default feed in a vector of 0's, but it must be of appropriate length to avoid syntax errors. Functions such as `calc_number_and_location_of_Ry_gates` serve this purpose.

λ parameter values:

We choose small values for λ to avoid accidentally overwhelming desirable ground states that naturally involve many gates and with energies that are of order 1 in the name of having less gates. In future iterations, penalizing the number of *layers* of R_y gates may be more optimal.

or layers of xy gates may be more optimal.

Number of gates to initialize with

We somewhat randomly initialize with 10 gates to start the search, however the mutations ensure this number will change wildly.

Mutation frequencies

A high mutation frequency is chosen to ensure adequate exploration of the extremely complex parameter space in the short number of generations of computational time we have available---however this may hurt convergence later on.

Elite individuals

We retain the best performer from the previous generation in the next generation, which ensures smooth always-improving tracking of performance. However, this does run the risk of the elite individual at a local minimum dominating the population through reproduction, which would negatively impact convergence to the global minimum. Again, this choice is motivated by the knowledge that we have few generations of computational time to work with.

Classical optimization trials to avoid local minima

Again to speed up computation in our limited computational time, we only re-run the classical optimization process 5 times per circuit, which increases the chances we find a false minimum set of θ_i .

```
1 def create_quantum_circuit(list_of_gates, list_of_theta_values): #Creates a qiskit quantum circuit obj
2     # print("Creating quantum circuit! Length of list of gates:", len(list_of_gates), "and list of thetas:", len(list_of_theta_values))
3     qc = qiskit.QuantumCircuit(n_qubits) #n_qubits is global variable
4     theta_counter = 0 #Will be used to deploy the theta variables in with the y gates only. Will increment by 1 for each y gate
5     for gate, qubits in list_of_gates:
6         if gate == "h":
7             qc.h(qubits[0])
8         elif gate == "x":
9             qc.x(qubits[0])
10        elif gate == "cx":
11            qc.cx(qubits[0], qubits[1])
12        elif gate == "y":
13            qc.y(qubits[0])
14            theta_counter += 1
15            list_of_theta_values.append(theta_counter * 2 * np.pi)
```

```
12     elif gate == 'ry':
13         qc.ry(list_of_theta_values[theta_counter], qubits[0])
14         theta_counter += 1
15     return qc
16
17 def create_list_of_thetas(list_of_gates): #Automatically detects how many rotational_y gates are in the
18     number_of_thetas = calc_number_and_location_of_Ry_gates(list_of_gates)[0]
19     list_of_thetas = np.random.uniform(-np.pi, np.pi, size=number_of_thetas)
20     return list_of_thetas
21
22 def calc_number_and_location_of_Ry_gates(list_of_gates): #Calculates the number of Ry gates in the circuit
23     total_number_of_y_gates = 0
24     list_of_indices_with_y_gates = []
25     for i in range(len(list_of_gates)):
26         if list_of_gates[i][0] == "ry":
27             total_number_of_y_gates += 1
28             list_of_indices_with_y_gates.append(i)
29     return total_number_of_y_gates, list_of_indices_with_y_gates #For now, second argument is redundant
30
31
32 def calculate_energy(list_of_theta_values, list_of_gates): #For a GIVEN value of theta, calculates <H>_E
33                                     #For circuits with Ry gates, we will optimize
34     qc = create_quantum_circuit(list_of_gates, list_of_theta_values)
35     psi = qiskit.quantum_info.Statevector.from_instruction(qc) #Computes the state prepared by that circuit
36     expval_energy = psi.expectation_value(H) #Built-in expectation value attribute of qiskit objects.
37     return np.real(expval_energy)
38
39 def calculate_ground_state_energy_and_theta(list_of_theta_values, list_of_gates): #Given a list of *in
40     number_of_theta_values = len(list_of_theta_values)
41     if number_of_theta_values == 0: #If there are no Ry gates, there is nothing to optimize. Just return
42         # print("There are no theta values/Ry gates in this circuit. Returning immediate energy.")
43         return calculate_energy(list_of_theta_values, list_of_gates), [] #The empty list matches the format
44
45     best_E = np.inf #Variable where the best energy will be located
46     best_result = None #and the best result output of the minimize method (also contains a redundant copy)
47
48     for trial in range(num_trials):
49         theta0 = np.random.uniform(
50
```

```
50         -np.pi,
51         np.pi,
52         size=number_of_theta_values)
53
54     result = minimize(
55         calculate_energy, #Function to be minimized, viewed as having parameters theta, and fixed a
56         theta0, #Initial guess
57         method="COBYLA",
58         args=list_of_gates)
59
60     if result.fun < best_E:
61         best_E = result.fun
62         best_result = result
63
64     ground_energy = result.fun
65     minimal_theta_array = result.x
66     return ground_energy, minimal_theta_array
67
68
69
70 class individual:
71     #Creates a class instance of a typical individual in the population. Each individual is represented
72     #Some methods are available to update attributes (namely the energy/fitness scores) upon, say, muta
73     def __init__(self, list_of_gates):
74         self.list_of_gates = list_of_gates
75         self.number_of_gates = len(list_of_gates)
76         self.number_of_Ry_gates = calc_number_and_location_of_Ry_gates(list_of_gates)[0]
77         self.ground_state_energy = None
78         self.fitness = None
79         self.best_theta_values = None
80     def calc_fitness(self):
81         list_of_gates = self.list_of_gates
82         list_of_thetas = create_list_of_thetas(list_of_gates)
83         ground_state_energy, best_theta_values = calculate_ground_state_energy_and_theta(list_of_thetas
84         self.ground_state_energy = ground_state_energy
85         self.best_theta_values = best_theta_values
86         self.fitness = fitness_score(list_of_gates, ground_state_energy)
87     def display(self):#NOTE- this will error if you have not yet computed a list_of_theta_values for th
```

```
88         #To remedy this, you must first use calc_fitness on the individual to fill in its best_theta_val
89         print(create_quantum_circuit(self.list_of_gates, self.best_theta_values))
90
91 def fitness_score(list_of_gates, ground_state_energy):    #User-defined fitness function. Generally, it
92     number_of_gates = len(list_of_gates)
93     number_of_Ry_gates = calc_number_and_location_of_Ry_gates(list_of_gates)[0]
94     fitness = - ground_state_energy - depth_penalty * number_of_gates - Ry_penalty * number_of_Ry_gates
95     return fitness
96
97
98 #####Genetic algorithm related function definitions
99 def tournament(individual_1, individual_2):    #Compares two individuals, selects one with higher fitness
100     fitness_1 = individual_1.fitness; fitness_2 = individual_2.fitness
101     if fitness_1 == None: #Calculates fitness if not done already
102         individual_1.calc_fitness()
103         fitness_1 = individual_1.fitness
104     if fitness_2 == None:
105         individual_2.calc_fitness()
106         fitness_2 = individual_2.fitness
107
108     if fitness_1 >= fitness_2:
109         # print("Individual 1 survives")
110         return individual_1
111     else:
112         # print("Individual 2 survives")
113         return individual_2
114
115 # survivor = tournament(my_individual, my_individual_2)
116 # print("Survivor:", survivor)
117 # survivor.display()
118
119
120 def mutate(individual): #Mutates a circuit by messing with its gates
121     #This will act via shallow copy, i.e. it will modify the genome in place!
122     genome = individual.list_of_gates
123     length_of_genome = len(genome)
124     if random.random() <= mutation_rate:
125         # print("Mutation occurring!")
```

```
126
127     ###1/3 probability for each mutation. Roll a dice and see what happens
128     which_mutation_random_number = random.random()
129     if which_mutation_random_number <= 0.3333: #Change gate
130         if length_of_genome == 0:
131             # print("WHOA NELLY!!!!\n\nMutation aborted- can't modify gate from a circuit with 0 gates")
132             pass
133         else:
134             gate_index_to_be_mutated = random.randint(0,length_of_genome-1)
135             # print("Gate change occurring at index", gate_index_to_be_mutated)
136             genome[gate_index_to_be_mutated] = random_gate_and_qubit() #Swaps current gate to be equal to a random gate
137     elif which_mutation_random_number <= 0.6666: #Add gate
138         gate_index_to_add_at = random.randint(0,length_of_genome)
139         # print("Gate addition occurring at index", gate_index_to_add_at)
140         genome.insert(gate_index_to_add_at, random_gate_and_qubit()) #Adds random gate at this index
141     else: #Delete gate
142         if length_of_genome == 0:
143             # print("WHOA NELLY!!!!\n\nMutation aborted- can't delete gate from a circuit with 0 gates")
144             pass
145         else: #Proceed
146             gate_index_to_delete_at = random.randint(0,length_of_genome-1)
147             # print("Gate deletion occurring at index", gate_index_to_delete_at)
148             genome.pop(gate_index_to_delete_at) #Deletes this gate
149             # print("If I made it here, the pop function is working properly.")
150         # print("Mutation complete. New genome:", genome)
151     else:
152         # print("No mutation occurring.")
153         pass
154
155
156 def random_gate_and_qubit(): #Helper function that creates a random gate somewhere in a circuit. Also returns the qubit(s) it acts on
157     random_gate_type = random.choice(["x", "h", "cx", "ry"])
158     if random_gate_type == "cx":
159         random_qubit = random.sample(range(0, n_qubits), 2) #Gets two different qubits for a cx gate
160     else:
161         random_qubit = [random.randint(0,n_qubits-1)]
162     # print("Random gate has been created for some purpose! Gate type and qubit:", (random_gate_type, random_qubit))
163     return (random_gate_type, random_qubit)
```

```
164
165
166 # mutate(my_individual)
167 # my_list_of_gates = my_individual.list_of_gates
168 # my_best_theta = my_individual.best_theta_values
169 # print("List of gates:", my_list_of_gates)
170 # print("Best theta values:", my_best_theta)
171
172 # qc = create_quantum_circuit(my_list_of_gates, my_best_theta)
173 # print(qc)
174
175 # my_individual.display()
176
177 def crossover(individual_1, individual_2): #Splices genes from two individuals to create a new individ
178     genome_1 = individual_1.list_of_gates; genome_2 = individual_2.list_of_gates
179     length_of_genome_1 = len(genome_1); length_of_genome_2 = len(genome_2)
180
181     limit_of_genome_1 = random.randint(0, length_of_genome_1)
182     limit_of_genome_2 = random.randint(0, length_of_genome_2)
183
184     new_genome = genome_1[:limit_of_genome_1] + genome_2[limit_of_genome_2:]
185
186     new_individual = individual(new_genome)
187     return new_individual
188
189 # new_individual = crossover(my_individual, my_individual_2)
190 # new_individual.calc_fitness()
191 # print("Genome:", new_individual.list_of_gates)
192
193 #####Functions relating to population generation/initialization#####
194
195 def create_random_list_of_gates(limit_on_number_of_gates):
196     list_of_gates = []
197     number_of_gates_to_use_for_this_individual = random.randint(0,limit_on_number_of_gates)
198     for i in range(0,number_of_gates_to_use_for_this_individual):
199         list_of_gates.append(random_gate_and_qubit())
200     return list_of_gates
201
```

```
202 # create_random_list_of_gates(5)
203
204 def create_population(population_size):
205     population = []
206     for i in range(population_size):
207         list_of_gates_to_add = create_random_list_of_gates(limit_on_number_of_gates)
208         # print("List of gates this individual will be initialized with:", list_of_gates_to_add)
209         an_individual = individual(list_of_gates_to_add)
210         population.append(an_individual)
211     return population
212
213 # population = create_population(initial_population_size)
214 #####
215
216 #####Population statistics#####
217
218 def find_best_individual(population):
219     best_fitness = -np.inf
220     best_individual = None
221     # print("Starting going through members!")
222     for member in population:
223         # print("Current best fitness I've seen so far:", best_fitness)
224         # print("Considering an individual")
225         # print("Looking at a member with fitness", member.fitness)
226         if member.fitness == None: #Calculates fitness if not already computed
227             # print("Individual had blank fitness. Recalculating.")
228             member.calc_fitness()
229
230         if member.fitness > best_fitness:
231             # print("This member's fitness of", member.fitness, "was better than the best fitness of",
232             best_individual = member
233             best_fitness = member.fitness
234         else:
235             # print("This member's fitness of", member.fitness, "was worse than the best fitness of", b
236             pass
237
238     return best_individual
239
```

```
240 # debug_individual = population[0]
241 # print("List of gates:", debug_individual.list_of_gates)
242 # best_individual = find_best_individual(population)
243
244 # print("Best individual found among initial population!")
245 # best_individual.display()
246 # print("Stats: Energy:", best_individual.ground_state_energy, "and fitness", best_individual.fitness)
247
248 def find_average_fitness(population):
249     fitness_scores = []
250     for individual in population:
251         # print("Considering an individual")
252         if individual.fitness == None: #Calculates fitness if not already computed
253             # print("Individual had blank fitness. Recalculating.")
254             individual.calc_fitness()
255             fitness_scores.append(individual.fitness)
256     average_fitness = np.mean(fitness_scores)
257
258     return average_fitness
259
260 #####
261
262 #####Global Variables#####
263 n_qubits = 10
264 H = transverse_ising_hamiltonian(n_qubits, J=1,h=1) #Hamiltonian to be minimized
265 depth_penalty = .01 #Penalty in fitness score for circuits with many gates
266 Ry_penalty = .01 #Penalty in fitness score specifically for Ry gates, which are discouraged.
267 num_trials = 5 ##Number of trials to perform during the classical minimization over theta values. Help
268
269 mutation_rate = .4 #Fraction of the time to mutate a child circuit (among these, evenly split addition
270
271 initial_population_size = 30 #Initial population of circuits
272 limit_on_number_of_gates = 10 #When we initialize the population, at most how many gates we allow in t
273
274 number_of_generations_to_run = 10 #Number of iterations of the entire algorithm to run
275 number_tournaments_each_generation = 30 #Number of pairwise tournaments to perform. Also equals number
276 num_children_each_generation = initial_population_size #Equals the population of the next generation. H
277 #####
```

```
278
279 # #####Example usage: finding the fitness score of an individual (note: n_qubits mus
280 # example_list_of_gates = [
281 #     ("h", [0]),          # Hadamard on qubit 0
282 #     ("x", [1]),          # X gate on qubit 1
283 #     ("cx", [0, 1]))      # CNOT with control 0 and target 1
284 #     # ("ry", [1]),
285 #     # ("ry", [3])]
286
287
288 # example_list_of_gates_2 = [
289 #     ("h", [0]),          # Hadamard on qubit 0
290 #     ("x", [1]),          # X gate on qubit 1
291 #     ("cx", [0, 1]),      # CNOT with control 0 and target 1
292 #     ("ry", [1]),
293 #     ("ry", [2])]
294
295 # print("Creating individual")
296 # my_individual = individual(example_list_of_gates)
297 # print("Applying calc_fitness method...")
298 # my_individual.calc_fitness()
299 # print("Application of method complete.\n")
300
301 # my_individual_2 = individual(example_list_of_gates_2)
302 # print("Applying calc_fitness method...")
303 # my_individual_2.calc_fitness()
304
305 # print("Ground state energy of my circuit topology:", my_individual.ground_state_energy)
306 # print("and best theta values for this circuit topology:", my_individual.best_theta_values)
307 # print("Fitness of this circuit topology:", my_individual.fitness)
308 # print("with circuit diagram"); print(create_quantum_circuit(my_individual.list_of_gates, my_individual
309
310 # print("Ground state energy of my circuit topology:", my_individual_2.ground_state_energy)
311 # print("and best theta values for this circuit topology:", my_individual_2.best_theta_values)
312 # print("Fitness of this circuit topology:", my_individual_2.fitness)
313 # print("with circuit diagram"); print(create_quantum_circuit(my_individual_2.list_of_gates, my_individual
```

```

1 #####MAIN SIMULATION#####
2 ###List and counter initializations
3 initial_population = create_population(initial_population_size)
4 print("Initial population initialized. Number of individuals:", len(initial_population))
5 current_population = copy.deepcopy(initial_population) #For the first iteration only, set the working po
6
7 current_tournament_winners = [] #Tournament winners and children will be stored here
8 future_children = []
9
10 generation_number = 1 #Counter for which generation we're on in the simulation
11 list_of_average_fitnesses = []
12 list_of_fitness_stddevs = []
13 list_of_best_individual_fitness = []
14
15 ###Main loop
16 for current_generation_number in range(number_of_generations_to_run): #Each run of this loop corresponds
17     print("###Generation", generation_number, "simulation starting now!")
18     current_population_size = len(current_population)
19
20     print("Extracting fitness scores of current population:")
21     list_of_current_fitness_scores = []
22     for member in current_population:
23         if member.fitness == None: #Calculates fitness if not already computed
24             # print("Individual had blank fitness. Recalculating.")
25             member.calc_fitness()
26         list_of_current_fitness_scores.append(member.fitness)
27     print("List of current fitness scores:", list_of_current_fitness_scores)
28     print("Standard deviation of current fitness scores:", np.std(list_of_current_fitness_scores))
29
30     ###Putting these statistics into a list
31     best_individual_of_current_gen = find_best_individual(current_population) #Will have a "free ticket"
32     best_fitness_of_current_gen = best_individual_of_current_gen.fitness
33     print("Best fitness of the current generation:", best_fitness_of_current_gen)
34     list_of_best_individual_fitness.append(best_fitness_of_current_gen)
35     list_of_average_fitnesses.append(find_average_fitness(current_population))
36     list_of_fitness_stddevs.append(np.std(list_of_current_fitness_scores))
37
38     print("Tournament stage:")

```

```
39     for i in range(number_tournaments_each_generation): #each i represents a single tournament
40         random_individual_1, random_individual_2 = random.choice(current_population), random.choice(current_population)
41         survivor = tournament(random_individual_1, random_individual_2)
42         current_tournament_winners.append(survivor) #Add winner of tournament to intermediary list of tournament winners
43     print("Tournament complete. Number of tournament winners:", len(current_tournament_winners))
44
45     print("Beginning crossovers:")
46     for i in range(num_children_each_generation):
47         i, j = random.randint(0, len(current_tournament_winners)-1), random.randint(0, len(current_tournament_winners)-1)
48         new_child = crossover(current_tournament_winners[i], current_tournament_winners[j])
49         future_children.append(new_child) #Append this child to the list of children
50     print("Crossovers completed! Number of children:", len(future_children))
51
52     print("Beginning mutations:")
53     mutation_counter = 0
54     for i in range(len(future_children)):
55         if random.random() <= mutation_rate: #If we get "lucky" enough to have a mutation:
56             mutate(future_children[i])
57             mutation_counter += 1
58         else:
59             pass
60     print("Mutations finished. Number of mutations conducted:", mutation_counter)
61
62     print("Almost done generating children- adding elite individual from current generation.")
63     future_children.pop(0); future_children.append(best_individual_of_current_gen) #Removes one child and adds elite
64     print("Generation of children complete!")
65
66
67     print("Setting parents of next generation equal to children of this generation, and clearing tournament winners")
68     current_population = copy.deepcopy(future_children)
69
70
71     current_tournament_winners = []
72     future_children = []
73     print("Complete.")
74
75     print("###Generation", generation_number, "complete.")
76     generation_number += 1 #Increment counter variable
```

```

77
78 print("###Main loop complete after", generation_number-1, "generations!")
79
80 print("Calculating fitness of final population:")
81 final_population = current_population
82 for member in final_population: #Since this generation is created outside of the loop, we must evaluate
83     member.calc_fitness()
84 fitness_scores_of_final_population = [member.fitness for member in final_population]
85 print("Fitness scores of final population:", fitness_scores_of_final_population)
86
87 list_of_best_individual_fitness.append(find_best_individual(final_population).fitness)
88 list_of_average_fitnesses.append(np.mean(fitness_scores_of_final_population))
89 list_of_fitness_stddevs.append(np.std(fitness_scores_of_final_population))
90
91 print("DONE--- SIMULATION COMPLETE")

```

```

Initial population initialized. Number of individuals: 30
###Generation 1 simulation starting now!
Extracting fitness scores of current population:
List of current fitness scores: [np.float64(8.36213595104714), np.float64(7.963525910116931), np.float64(6.999999999999999), np.float64(9.394435328611369), np.float64(9.79864889138539)]
Standard deviation of current fitness scores: 2.320647724477552
Best fitness of the current generation: 9.394435328611369
Tournament stage:
Tournament complete. Number of tournament winners: 30
Beginning crossovers:
Crossovers completed! Number of children: 30
Beginning mutations:
Mutations finished. Number of mutations conducted: 13
Almost done generating children- adding elite individual from current generation.
Generation of children complete!
Setting parents of next generation equal to children of this generation, and clearing tournament and children
Complete.
###Generation 1 complete.
###Generation 2 simulation starting now!
Extracting fitness scores of current population:
List of current fitness scores: [np.float64(6.99), np.float64(4.979999999999999), np.float64(9.394213561252001), np.float64(9.79864889138539), np.float64(9.79864889138539)]
Standard deviation of current fitness scores: 2.324569900714548
Best fitness of the current generation: 9.79864889138539
Tournament stage:
Tournament complete. Number of tournament winners: 30

```

```

Tournament complete. Number of tournament winners: 30
Beginning crossovers:
Crossovers completed! Number of children: 30
Beginning mutations:
Mutations finished. Number of mutations conducted: 13
Almost done generating children- adding elite individual from current generation.
Generation of children complete!
Setting parents of next generation equal to children of this generation, and clearing tournament and children
Complete.
###Generation 2 complete.
###Generation 3 simulation starting now!
Extracting fitness scores of current population:
List of current fitness scores: [np.float64(6.004716869756639), np.float64(5.520281536352328), np.float64(8.
Standard deviation of current fitness scores: 1.8338774806535254
Best fitness of the current generation: 9.79864889138539
Tournament stage:
Tournament complete. Number of tournament winners: 30
Beginning crossovers:
Crossovers completed! Number of children: 30
Beginning mutations:
Mutations finished. Number of mutations conducted: 15
Almost done generating children- adding elite individual from current generation.
Generation of children complete!
Setting parents of next generation equal to children of this generation, and clearing tournament and children
Complete.
###Generation 3 complete.
###Generation 4 simulation starting now!
Extracting fitness scores of current population:
List of current fitness scores: [np.float64(2.7886488818524318), np.float64(7.989999999999999), np.float64(8
Standard deviation of current fitness scores: 2.266953839870687
Best fitness of the current generation: 10.003747681152827
Tournament stage:
Tournament complete. Number of tournament winners: 30
Beginning crossovers:
Crossovers completed! Number of children: 30

```

```
1 print("Fitness scores of final population:", fitness_scores_of_final_population)
```

```
Fitness scores of final population: [np.float64(9.610281539787161), np.float64(7.35213595411452), np.float64
```

Visualization of Results: Best individual produced after simulation and progression

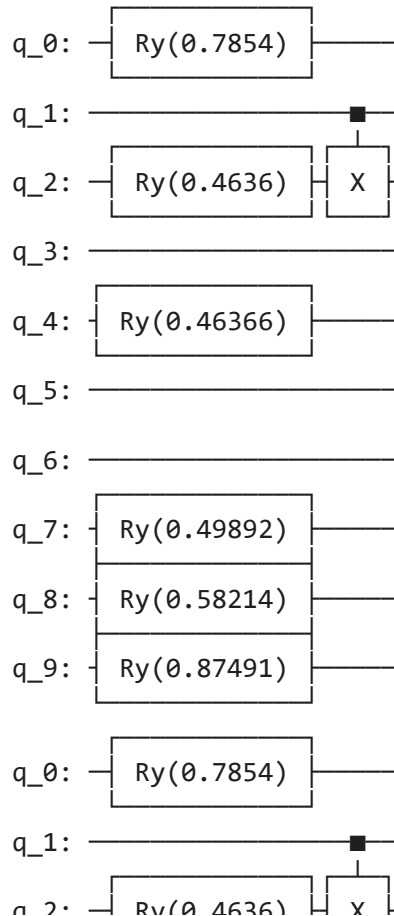
- ✓ visualization of results. Best individual produced after simulation, and progression of average fitness as generations progressed.

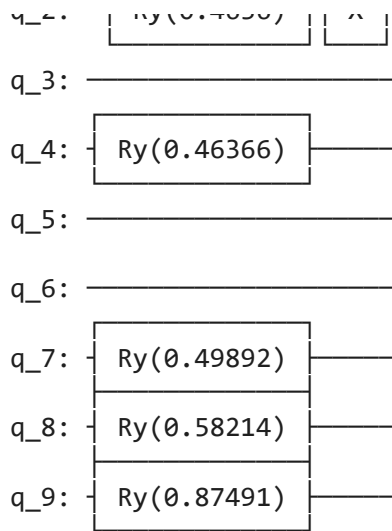
```

1 best_individual = find_best_individual(current_population)
2 print("Best individual found among initial population!")
3 best_individual.display()
4
5 print(create_quantum_circuit(best_individual.list_of_gates, best_individual.best_theta_values))
6
7 print("Stats: Energy:", best_individual.ground_state_energy, "and fitness", best_individual.fitness)

```

Best individual found among initial population!





Stats: Energy: -10.829166212415727 and fitness 10.699166212415726

✓ Metrics of Circuit Energy/Fitness Over Time

```

1 ###Plot average fitness (and stddev of fitnesses) over time
2 x = np.array(range(0,number_of_generations_to_run+1))
3 true_energy = [12.381489999654757]*(number_of_generations_to_run+1)
4
5 plt.figure()
6 plt.title("Average and Elite Fitness vs. Generation\nfor Genetically-Produced Quantum Circuits Preparin
7 plt.plot(x, list_of_average_fitnesses, 'k-', label = "Average Individual", color = "green")
8 plt.fill_between(x, np.array(list_of_average_fitnesses)-np.array(list_of_fitness_stddevs), np.array(lis
9                     alpha=0.3, facecolor='green', label = "1 standard deviation")
10 plt.plot(list_of_best_individual_fitness, label = "Elite Individual", color="purple")
11 plt.plot(true_energy, label = "Analytical Energy", linestyle='dashed', color="black")
12 plt.grid()
13 plt.xlabel("Generation")
14 plt.ylabel("Fitness Score")
15 plt.legend()
16 plt.ylim(0,12.5)
17 plt.show()

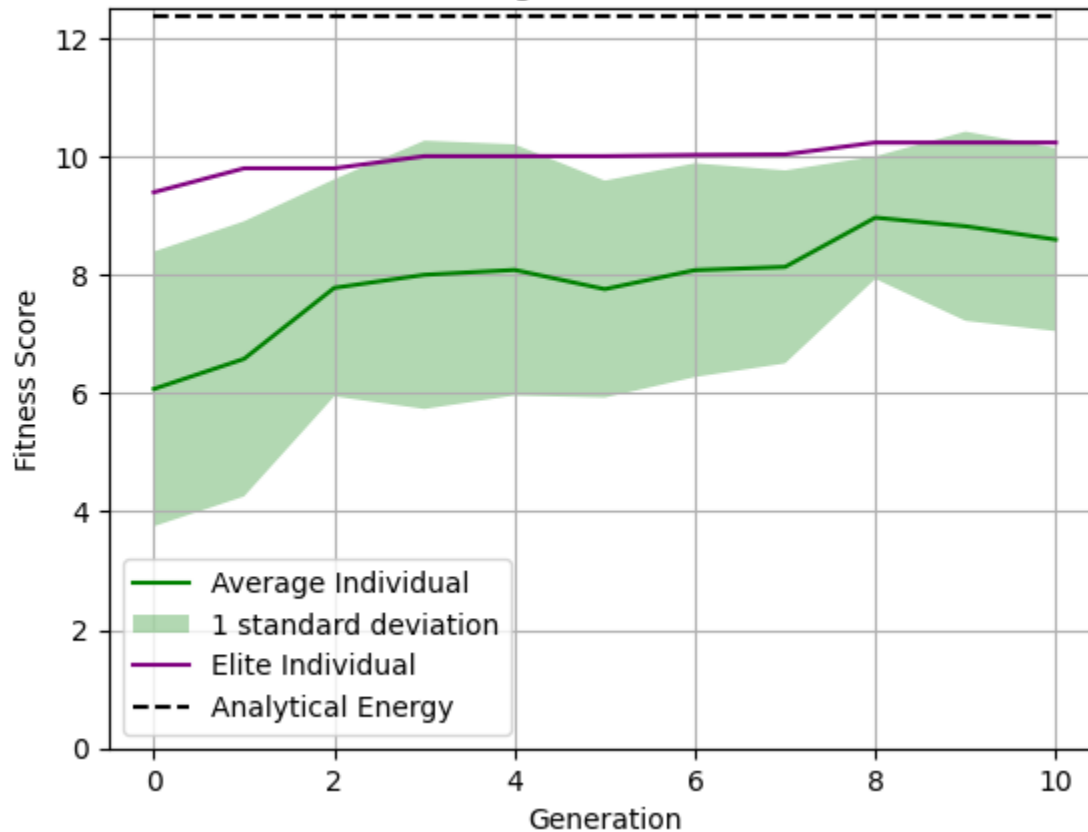
```

```

<>:7: SyntaxWarning: invalid escape sequence '\s'
<>:7: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_76779/2033548927.py:7: SyntaxWarning: invalid escape sequence '\s'
  plt.title("Average and Elite Fitness vs. Generation\nfor Genetically-Produced Quantum Circuits Preparing G
/tmp/ipykernel_76779/2033548927.py:8: UserWarning: color is redundantly defined by the 'color' keyword argum
  plt.plot(x, list_of_average_fitnesses, 'k-', label = "Average Individual", color = "green")

```

Average and Elite Fitness vs. Generation
for Genetically-Produced Quantum Circuits Preparing Ground State
of the $n = 10$ Ising Model (with 1σ error bars)



If above figure is blank, see figure in introduction! (Printing to PDF can destroy Matplotlib figures)

✓ Brief Conclusions

The algorithm was a proof-of-concept for the design of a genetic algorithm capable of selecting for circuits identifying the ground-state eigenvectors/eigenvalues of the transverse Ising Hamiltonian.

In our short test of only 10 generations (limited by computation time on an online Colab notebook---a proper run on actual hardware could be run much longer and with a much more substantial population if desired), the algorithm identified a circuit capable of identifying the minimum eigenvalue to within 15% of the true eigenvalue (~10.8 depending on random seeding, vs ~12.5 exact result) for a 10-qubit system. (Already for 20 qubits the Hamiltonian cannot be stored in a performance laptop's RAM, so an analytical answer via exact diagonalization is not available for a general H of this size.) Moreover, it did so in a gradient-free way, meaning that we were able to optimize in a way that did not rely on any notion of continuously-variable parameters, instead with us being able to delete or modify discrete quantum gates.

The VQE is a promising approach for diagonalizing extremely large matrices, especially those appearing in chemistry/physical systems which have sparse representations in terms of Pauli strings, meaning that few physical qubit measurements will be required and allowing for meaningful quantum speedup.

Natural next steps would be to vary which gate families are allowed (e.g. upgrade to a universal set of gates or restrict to gates most likely to generate an approximate solution, based off of the real-ity properties of a given Hamiltonian), or to integrate high-performance computing into the classical optimization steps, which still occupy a bulk of the computational time.

This investigation represents a domain of very meaningful interplay and co-integration of classical computing, quantum computing, and machine learning into one physically-meaningful application.

1 ###Nicholas Pittman 6-24-2026

